Phase 3: Multi-Agent System Architecture Design Proposal for Tau-Bench

Team TLDR

February 26, 2026

1 Introduction

Based on Phase 1 and 2 failure logs, single-instance LLMs often fail due to loop repetitions, context hallucinations, and constraint violations. We propose a modular LangGraph-orchestrated Multi-Agent Architecture to directly address our 6 failure categories by assigning specialized nodes that intercept errors before environmentally impacting the task.

2 Error categories observed

The list below contains the error categories that were observed during our analysis of the Phase 1's JSON results.

- **Tool Parameter Errors**
- **Missing Preconditions**
- **Context Identity Errors**
- **Task Abandonment**
- **Business Logic Errors**
- **Repetitive Stuck Loops**
- **Others**

3 Proposed System Architecture

Our framework replaces monolithic prompting with multi-agent orchestration, incorporating three industry-standard paradigms to strictly separate concerns:

1. **Hierarchical Supervisor-Worker:** A "Supervisor" dictates high-level objectives, delegating the actual tool call drafting to an "Executor."
2. **Actor-Critic Feedback:** The "Executor" (Actor) generates raw actions, while a "Validator" (Critic) injects self-correction feedback if those proposed actions breach business policies.
3. **Plan-then-Execute (PtE):** Segregating planning from execution prevents cognitive overload. It stops the agent from overanalyzing a situation to the point of inaction, and naturally breaks repetitive error loops.

This structurally guarantees dynamic verification of constraints and schemas prior to execution. Below is our component breakdown.

3.1 Architecture Diagram

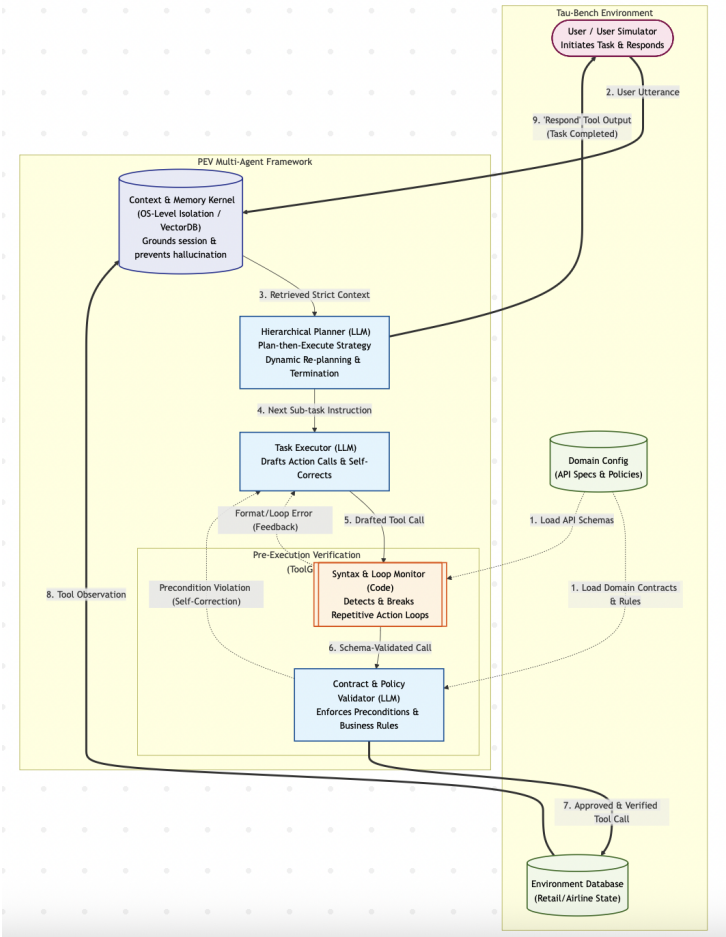

Figure 1: Proposed Architecture Diagram

4 Resolving the 6 Discovered Error Categories

This architecture incorporates bounded execution (e.g., ToolGate contracts, Plan-then-Execute) to resolve the Phase 2 failure categories, seamlessly plugging into Tau-Bench.

4.1 Tool Parameter Errors

The Failure: The agent fabricated parameters, dropped required prefixes, or used malformed IDs.

The Solution: Syntax & Loop Monitor (Code). This deterministic layer strict-checks drafted JSON against API schemas. Malformed arguments trigger an immediate self-correction feedback loop.

4.2 Missing Preconditions

The Failure: Attempting to book reservations without checking availability.

The Solution: Validator (LLM). Treating API tools as rigid contracts, this node confirms required prior observations exist in memory before allowing database writes.

4.3 Context Identity Errors

The Failure: Hallucinating user identities or importing cross-session contexts (e.g., referencing "Mia Li" when no name was provided).

The Solution: Context & Memory Kernel. Delegating conversation history to an OS-level session store (VectorDB) restricts the LLM exclusively to verified context chunks, eliminating contamination.

4.4 Task Abandonment

The Failure: Giving up after single failures, outputting `###STOP###` prematurely, or falsely claiming completion.

The Solution: Hierarchical Planner (LLM). Using a Plan-then-Execute methodology, the Supervisor maintains strategic goals and forces the Executor back to work until observations confirm completion.

4.5 Business Logic Errors

The Failure: Unauthorized operations, like processing refunds to incorrect payment methods or unchecked gift cards.

The Solution: Validator (LLM). This node evaluates actions against domain policies and returns unauthorized operations to the Executor for re-planning.

4.6 Repetitive Stuck Loops

The Failure: Repeatedly issuing the identical failed JSON payload after an API error.

The Solution: Syntax & Loop Monitor & Hierarchical Planner. If identical cyclic actions are detected, the monitor intercepts the loop and escalates to the Planner for dynamic re-planning.

5 Tau-Bench Evaluation and Integration Framework

Because Tau-Bench operates as an environmental simulator, the Plan Execute and Validate (PEV) framework effortlessly spans the project's experimental matrices:

- **Dual Domains (Airline & Retail):** The unified *Validator* dynamically loads corresponding API Specs & Policies from the Tau-Bench configuration.
- **4 Qwen3 Models (4B, 8B, 14B, 32B):** The specific LLM powering the Planner, Executor, and Validator nodes can be interchanged to evaluate baseline capabilities modularly.
- **3 Tool-Calling Methods (Act, ReAct, FC):** PEV accommodates Executor-level prompting baselines (FC, ReAct, Act) while maintaining identical pre-execution verification.
- **Pass^k Consistency Metrics (1 $\rightarrow$ 5):** Final multi-agent JSON traces pass through existing metric scripts, calculating probabilities identically to single-agent baselines.